CO3-AT2_Questions-Slot-C

Register No: 192311369
Student: ADITHYA R

Assigned Question:

Q1. Case Study: E-Commerce Product Sorting System (Merge Sort)

An e-commerce platform must sort millions of products based on price and ratings to ensure fast user access. Analyze how Merge Sort can be used to handle large datasets efficiently. Clearly identify key issues such as stability and scalability. Apply divide and conquer concepts to explain the sorting process. Analyze its time complexity and performance. Design a feasible solution and justify why Merge Sort is suitable for this scenario.

Solution :

1. Understanding the Problem

- An e-commerce platform holds millions of product records, each with attributes like price and customer rating.
- When a user applies a filter (for example, "sort by price: low to high" or "sort by rating: high to low"), the system must sort this huge dataset quickly and return results without noticeable delay.
- The sorting algorithm chosen must therefore be efficient at large scale, predictable in performance, and able to handle data that may not fit entirely in memory at once.

2. Applying Divide and Conquer: How Merge Sort Works

- Merge Sort follows the divide and conquer strategy, which breaks a large problem into smaller, easier sub-problems, solves them, and combines the results.
- Divide: The list of products is repeatedly split into two halves until each sub-list contains just a single product (a single element is always considered sorted).
- Conquer: Each small sub-list is trivially "sorted" since it has only one element.
- Combine (Merge): Pairs of sorted sub-lists are merged back together in order, comparing the front elements of each list and picking the smaller (or higher-rated) one first, until one fully sorted list remains.

Simplified Process for Product Sorting by Price:

```
def merge_sort(products):
    if len(products) <= 1:
        return products

    mid = len(products) // 2
    left = merge_sort(products[:mid])   # Divide
    right = merge_sort(products[mid:])  # Divide

    return merge(left, right)           # Conquer + Combine

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i]['price'] <= right[j]['price']:
            result.append(left[i]); i += 1
        else:
```

```
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

3. Stability: Why It Matters Here

- A sorting algorithm is stable if it keeps the original relative order of elements that have equal keys.
- Merge Sort is a stable algorithm because during the merge step, when two elements are equal, the one from the left sub-list is always placed first, preserving their original order.
- For an e-commerce platform, this matters when sorting by one field after already sorting by another. For example, if products are first sorted alphabetically and then sorted by price, a stable sort ensures that products with the same price still appear in alphabetical order within that price group, giving users a consistent and predictable browsing experience.

4. Scalability: Handling Millions of Products

- Merge Sort's divide and conquer approach naturally splits the workload into independent smaller tasks, which makes it well suited for large-scale and distributed systems.
- The sub-lists created during the divide step can be sorted in parallel across multiple threads, cores, or even different servers, and later merged together, which is essential for handling millions of products efficiently.
- Unlike in-place algorithms such as Quick Sort, Merge Sort's performance does not degrade on already sorted or nearly sorted data, giving it consistent and predictable behavior regardless of how the incoming product data is arranged.
- Merge Sort is also well suited to external sorting, where data is too large to fit in memory. Large chunks of product data can be sorted separately on disk and then merged, which is a common real-world requirement for e-commerce catalogs.

5. Time Complexity and Performance Analysis

- Divide Step: The list is split in half at each level, producing log(n) levels of splitting.
- Merge Step: At each level, merging all sub-lists back together takes O(n) time in total.
- Combining both steps gives an overall time complexity of O(n log n) for best, average, and worst cases, since Merge Sort always performs the same amount of splitting and merging regardless of the initial arrangement of the data.
- Space Complexity: O(n), because merging requires temporary arrays to hold elements during the combine step.

Complexity Summary:

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)
- Space Complexity: O(n)
- Stability: Stable

6. Proposed Solution Design

- Pre-sort catalog data periodically (e.g., every few minutes) using Merge Sort on separate fields such as price and rating, storing the sorted results as indexes.

- Split the product dataset into chunks and sort each chunk in parallel across multiple servers, then merge the sorted chunks together, taking advantage of Merge Sort's parallel-friendly structure.
- For real-time filtering or search queries, combine these pre-sorted indexes with fast merging, rather than sorting the entire dataset from scratch on every user request.
- Use external merge sort techniques for extremely large catalogs that cannot fit into memory, sorting data in manageable blocks on disk and merging the results.

Why Merge Sort Is Suitable for This Scenario

Merge Sort is a strong fit for an e-commerce product sorting system because it guarantees O(n log n) performance no matter how the input data is arranged, avoiding the unpredictable worst-case slowdowns that algorithms like Quick Sort can face. Its divide and conquer structure naturally supports parallel processing across multiple servers, which is essential when sorting millions of products at scale. It is also stable, ensuring that products with equal prices or ratings retain a consistent, predictable order, which improves user experience. Finally, Merge Sort works well with external sorting techniques, making it practical for catalogs too large to fit entirely in memory. Together, these properties make Merge Sort a reliable and scalable choice for this use case.